CSA0617-Design and Analysis of Algorithms

Name: Y.Thanushma

Registration Number:192425355

Day-1 Programmes

Experiment 1: Linear Search (Iterative)

Aim: To analyze the time complexity of the Linear Search algorithm.

Algorithm:

1. Start the program.
2. Define a list (array) of elements.
3. Take the key element to be searched.
4. Traverse the array from the first element to the last.
5. Compare each element with the key:
6. If a match is found, display the index and stop.
7. If the key is not found after checking all elements, display "Key not found".
8. End the program.

Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Output:

```
========================================
Key found at index 2
```

Experiment 2: Binary Search (Recursive)

Aim: To analyze and compare the time complexity of the Recursive Binary Search algorithm.

Algorithm:

1. Start the program.

2. Define a sorted array of elements.
3. Take the key element to be searched.
4. Define a recursive function with parameters: array, low index, high index, and key.
5. Find the middle element:
    - mid = (low + high) // 2
6. Compare the key with the middle element:
    - If equal → return index
    - If key < mid → search left subarray
    - If key > mid → search right subarray
7. Repeat recursively until:
    - Key is found, or
    - Low > High (element not found)
8. Display the result.
9. End the program.

Code:

```
def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)

    return -1

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr) - 1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Output:

```
====================================
Key found at index 3
```

Experiment 3: Factorial (Iterative vs Recursive)

Aim: To compare iterative and recursive methods for computing the factorial of a number and analyze their time complexity.

Algorithm:

Iterative Method

1. Start the program.
2. Input a number n.

3. Initialize fact = 1.
4. Use a loop from 1 to n.
5. Multiply each number with fact.
6. Display the result.
7. End the program.

Recursive Method

1. Start the program.
2. Input a number n.
3. Define a recursive function:
    - If n == 0 or n == 1, return 1.
    - Else return n * factorial(n-1).
4. Call the function and display the result.
5. End the program.

Code:

```
File  Edit  Format  Run  Options  Window  Help
def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)

n = 5
iter_result = factorial_iterative(n)
rec_result = factorial_recursive(n)
print("Iterative Result:", iter_result)
print("Recursive Result:", rec_result)
```

Output:

```
==============================
Iterative Result: 120
Recursive Result: 120
```

Experiment 4: Fibonacci Series

Aim: To analyze and compare recursive and iterative methods for generating the Fibonacci series.

Algorithm:

1. Input the value of n.
2. Iterative Method:
3. Initialize a = 0, b = 1.

4. Generate next terms using a loop (a + b) until n terms.
5. Recursive Method (Naïve):
6. Define function fib(n) = fib(n-1) + fib(n-2) with base cases 0 and 1.
7. Call function for each term.
8. Memoization Method:
9. Store computed Fibonacci values in a dictionary.
10. Reuse stored values to avoid recomputation.
11. Display all generated Fibonacci series.

Code:

```
def fib_iterative(n):
    series = []
    a, b = 0, 1
    for i in range(n):
        series.append(a)
        a, b = b, a + b
    return series
def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)
def generate_recursive(n):
    return [fib_recursive(i) for i in range(n)]
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
def generate_memo(n):
    return [fib_memo(i) for i in range(n)]
n = 6
print("Iterative:", fib_iterative(n))
print("Recursive (Naïve):", generate_recursive(n))
print("Recursive (Memoization):", generate_memo(n))
```

Output:

```
================================================
Iterative: [0, 1, 1, 2, 3, 5]
Recursive (Naïve): [0, 1, 1, 2, 3, 5]
Recursive (Memoization): [0, 1, 1, 2, 3, 5]
```

Experiment 5: Matrix Multiplication

Aim: To analyze the time complexity of matrix multiplication using nested loops.

Procedure:

1. Input two matrices A and B.
2. Check if multiplication is possible (columns of A = rows of B).
3. Initialize result matrix with zeros.
4. Use three nested loops:
5. Outer loop → rows of A
6. Middle loop → columns of B
7. Inner loop → multiply and add elements
8. Store results in the result matrix.

9. Display the final matrix.

Code:

Output:

```
Result: [[19, 22], [43, 50]]
```

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]


result = [[0, 0],
          [0, 0]]

for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            result[i][j] += A[i][k] * B[k][j]

print("Result:", result)
```

Experiment 6: Merge Sort

Aim: To analyze the divide-and-conquer approach in Merge Sort.

Procedure:

1. Input the array.
2. Divide the array into two halves recursively.
3. Continue dividing until single elements remain.
4. Merge the divided parts in sorted order.
5. Combine all parts to get the final sorted array.

Code:

Output:

```
File  Edit  Format  Run  Options  Window  Help
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

```
================================================
Sorted array: [3, 9, 10, 27, 38, 43, 82]
```

Experiment 7: Quick Sort

Aim: To analyze the performance of the recursive Quick Sort algorithm.

Algorithm:

1. Input the array.
2. Choose a pivot element.
3. Partition the array into:
4. Elements less than pivot
5. Elements greater than pivot
6. Recursively apply Quick Sort to both partitions.
7. Combine results to get the sorted array.

Code:

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

arr = [10, 7, 8, 9, 1, 5]

sorted_arr = quick_sort(arr)

print("Sorted array:", sorted_arr)
```

Output:

```
==================================
Sorted array: [1, 5, 7, 8, 9, 10]
```

Experiment 8: Tower of Hanoi

Aim: To analyze exponential recursion using the Tower of Hanoi problem.

Procedure:

1. Input number of disks n.
2. Define three rods: Source (A), Auxiliary (B), Destination (C).
3. Use recursion:
4. Move n-1 disks from Source to Auxiliary.
5. Move the nth disk from Source to Destination.
6. Move n-1 disks from Auxiliary to Destination.
7. Repeat until all disks are moved.
8. Display the sequence of moves.

Code:

daa8.py - C:/Users/Thanushma/Documents/daa8.py (3.14.3)

File Edit Format Run Options Window Help

```
def hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    hanoi(n-1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    hanoi(n-1, auxiliary, source, destination)

n = 3

hanoi(n, 'A', 'B', 'C')
```

Output:

```
==============================
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Experiment 9: GCD (Euclidean Algorithm)

Aim: To analyze the efficiency of the Euclidean algorithm using recursion.

Procedure:

1. Input two numbers a and b.
2. If b == 0, return a as GCD.
3. Otherwise, compute gcd(b, a % b).
4. Repeat recursively until remainder becomes 0.
5. Display the GCD.

Code:

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a = 48
b = 18

result = gcd(a, b)

print("GCD:", result)
```

Output:

```
--------------------------
GCD: 6
```

Experiment 10: Insertion Sort

Aim: To analyze the performance of the iterative Insertion Sort algorithm.

Procedure:

1. Input the array.
2. Start from the second element.

3. Compare it with previous elements.
4. Shift larger elements to the right.
5. Insert the element at correct position.
6. Repeat until the array is sorted.

Code:

```
arr = [12, 11, 13, 5, 6]

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted array:", arr)
```

Output:

```
====================================
Sorted array: [5, 6, 11, 12, 13]
```

Experiment 11: Heap Sort

Aim: To analyze the performance of heap-based sorting using Heap Sort.

Algorithm:

1. Input the array.
2. Build a max heap from the array.
3. Swap the first element (largest) with the last element.
4. Reduce heap size and heapify the root.
5. Repeat until the array is sorted.

Code:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)

arr = [4, 10, 3, 5, 1]

heap_sort(arr)

print("Sorted array:", arr)
```

Output:

```
========================================
Sorted array: [1, 3, 4, 5, 10]
```

Experiment 12: Counting Inversions

Aim: To analyze the use of a modified Merge Sort for counting inversions.

Algorithm:

1. Input the array.
2. Divide the array into two halves (like Merge Sort).
3. Recursively count inversions in left and right halves.
4. During merge, count cross inversions when elements from right are placed before left.
5. Add all inversion counts and display result.

Code:

```
def merge_count(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2
    left, inv_left = merge_count(arr[:mid])
    right, inv_right = merge_count(arr[mid:])

    merged = []
    i = j = inv_count = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            inv_count += len(left) - i
            j += 1

    merged += left[i:]
    merged += right[j:]

    return merged, inv_left + inv_right + inv_count

arr = [2, 4, 1, 3, 5]

_, result = merge_count(arr)

print("Number of inversions:", result)
```

Output:

```
==============================
Number of inversions: 3
```

Experiment 13: Maximum Subarray Sum

Aim: To compare Kadane's algorithm and Divide-and-Conquer approach for finding maximum subarray sum.

Algorithm:

1. Input the array.
2. eTraverse array, keep current sum and maximum sum.
3. Divide & Conquer:
4. Divide array into two halves.
5. Find max subarray in left, right, and crossing middle.
6. Compare results and display maximum sum.

Code:

```
File  Edit  Format  Run  Options  Window  Help
def kadane(arr):
    max_sum = arr[0]
    current_sum = arr[0]

    for i in range(1, len(arr)):
        current_sum = max(arr[i], current_sum + arr[i])
        max_sum = max(max_sum, current_sum)

    return max_sum
def max_crossing_sum(arr, left, mid, right):
    left_sum = float('-inf')
    total = 0
    for i in range(mid, left-1, -1):
        total += arr[i]
        left_sum = max(left_sum, total)

    right_sum = float('-inf')
    total = 0
    for i in range(mid+1, right+1):
        total += arr[i]
        right_sum = max(right_sum, total)

    return left_sum + right_sum

def max_subarray_dc(arr, left, right):
    if left == right:
        return arr[left]

    mid = (left + right) // 2

    return max(
        max_subarray_dc(arr, left, mid),
        max_subarray_dc(arr, mid+1, right),
        max_crossing_sum(arr, left, mid, right)
    )
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
print("Kadane's Result:", kadane(arr))
print("Divide & Conquer Result:", max_subarray_dc(arr, 0, len(arr)-1))
```

Output:

```
==========
Kadane's Result: 7
Divide & Conquer Result: 7
>>
```

Experiment 14: Exponentiation

Aim: To compare iterative and recursive (fast power) methods for computing exponentiation.

Algorithm:

1. Input values x and n.

Iterative Method:

2. Multiply x repeatedly n times.
3. Recursive Fast Power:
4. If n == 0, return 1.
5. If n is even → compute .If n is odd → compute
6. Display results.

Code:

```
File  Edit  Format  Run  Options  Window  Help

def power_iterative(x, n):
    result = 1
    for _ in range(n):
        result *= x
    return result

def power_fast(x, n):
    if n == 0:
        return 1
    half = power_fast(x, n // 2)
    if n % 2 == 0:
        return half * half
    else:
        return x * half * half

x = 2
n = 10
print("Iterative Result:", power_iterative(x, n))
print("Fast Power Result:", power_fast(x, n))
```

Output:

```
==============================
Iterative Result: 1024
Fast Power Result: 1024
```

Experiment 15: Closest Pair of Points

Aim: To analyze the difference between brute force and divide-and-conquer approaches for finding the closest pair of points.

Algorithm:

1. Input list of points.
2. Brute Force Method:
    - Compute distance between every pair.
    - Track minimum distance.
3. Divide & Conquer:
    - Sort points by x-coordinate.
    - Divide into two halves.
    - Recursively find closest pair in each half.
    - Check points near dividing line.
4. Compare results and display closest pair and distance.

Code:

File Edit Format Run Options Window Help

```
import math

def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def brute_force(points):
    min_dist = float('inf')
    pair = None
    n = len(points)

    for i in range(n):
        for j in range(i+1, n):
            d = distance(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])

    return pair, min_dist

points = [(1,2), (4,5), (7,8), (3,1)]

pair, dist = brute_force(points)

print("Closest pair:", pair)
print("Distance:", dist)
```

Output:

```
==================================
Closest pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
```